



Model Test Results Report

Automated Test Documentation for Model Governance

MKM Research Labs

Version 1.0 — 15-May-2026

David K Kelly

CONFIDENTIAL — PROPRIETARY

Legal Notice

PROPRIETARY AND CONFIDENTIAL

This document, including all algorithms, models, methodology, formulas, calculations, and intellectual concepts contained herein, constitutes the exclusive intellectual property and confidential trade secrets of MKM Research Labs (“MKM”).

Any usage, reproduction, distribution, modification, or disclosure of this document or any portion thereof, without the express prior written authorisation from MKM Research Labs is strictly prohibited and constitutes an infringement of intellectual property rights.

The document is provided on an “as is” basis. MKM Research Labs makes no representations or warranties of any kind, express or implied, regarding its accuracy, reliability, or suitability for any particular purpose.

All rights reserved. © 2021–2026 MKM Research Labs.

Governed by the laws of the United Kingdom.

Contents

1 Executive Summary 3

1.1 Test Environment 3

1.2 Results by Model 3

2 Test Traceability Matrix 4

3 Building Resilience Index Model (MKM-BRI-001) 10

1 Executive Summary

This report documents the results of automated model tests run on **15 May 2026 at 12:01:31**.

Table 1: Overall Test Results

Total Tests	202
Passed	202
Failed	0
Skipped	0
Pass Rate	100.0%
Total Duration	5.63s

1.1 Test Environment

Platform	macOS-26.4.1-arm64-arm-64bit
Python	3.9.6
pytest	8.4.2

1.2 Results by Model

Table 2: Test Results by Model

Model ID	Model Name	Total	Pass	Fail	Skip	Time (s)
MKM-BRI-001	Building Resilience Index Model	202	202	0	0	0.21

2 Test Traceability Matrix

The following table maps every executed test to its assigned model component, providing evidence of coverage required under SR 11-7 §4.2 and SS1/23.

Model	Module	Test Name	Status
MKM-BRI-001	test_bri_aggregation.py::TestCalibration	test_calibrate_aggregation_summary_counts	Pass
	test_bri_aggregation.py::TestCalibration	test_calibrate_aggregation_summary_empty	Pass
	test_bri_aggregation.py::TestCalibration	test_calibrate_aggregation_rate_empty_returns_current	Pass
	test_bri_aggregation.py::TestCalibration	test_calibrate_aggregation_rate_thresholds_hits_target	Pass
	test_bri_aggregation.py::TestContingency	test_contingency_assessed_is_NR	Pass
	test_bri_aggregation.py::TestContingency	test_contingency_verified_no_hazard_profile_is_toprating	Pass
	test_bri_aggregation.py::TestContingency	test_contingency_empty_input	Pass
	test_bri_aggregation.py::TestContingency	test_contingency_hazards_excluded_from_weakest_link	Pass
	test_bri_aggregation.py::TestContingency	test_contingency_modifier_not_applied_to_nr	Pass
	test_bri_aggregation.py::TestContingency	test_contingency_modifier_when_continuity_strong	Pass
	test_bri_aggregation.py::TestContingency	test_contingency_full_breakdown	Pass
	test_bri_aggregation.py::TestContingency	test_contingency_single_link_picks_worst_hazard	Pass
	test_bri_aggregation.py::TestField	test_field_basement_flood_strategy[Deliberate-floodable with protection-0.75]	Pass
	test_bri_aggregation.py::TestField	test_field_basement_flood_strategy[Flood-resistant basement-1.0]	Pass
	test_bri_aggregation.py::TestField	test_field_basement_flood_strategy[No basement-1.0]	Pass
	test_bri_aggregation.py::TestField	test_field_basement_flood_strategy[None-0.0]	Pass
	test_bri_aggregation.py::TestField	test_field_boolean_false_zero_credit	Pass
	test_bri_aggregation.py::TestField	test_field_boolean_true_full_credit	Pass
	test_bri_aggregation.py::TestField	test_field_none_value_is_zero	Pass
	test_bri_aggregation.py::TestField	test_field_resilience_levels[Enhanced-0.9]	Pass
	test_bri_aggregation.py::TestField	test_field_resilience_levels[Meets minimum-0.7]	Pass
	test_bri_aggregation.py::TestField	test_field_resilience_levels[Not assessed-0.0]	Pass
	test_bri_aggregation.py::TestField	test_field_resilience_levels[Partial-0.4]	Pass
	test_bri_aggregation.py::TestField	test_field_resilience_levels[Verified-1.0]	Pass
	test_bri_aggregation.py::TestField	test_field_unknown_string_zero	Pass
	test_bri_aggregation.py::TestMap	test_map_to_threshold	Pass
	test_bri_aggregation.py::TestMap	test_map_to_rating	Pass
	test_bri_aggregation.py::TestMap	test_map_to_rating	Pass
	test_bri_aggregation.py::TestMap	test_map_to_rating	Pass
	test_bri_aggregation.py::TestMap	test_map_to_rating	Pass
	test_bri_aggregation.py::TestScore	test_score_hazard_with_all_verified_is_one	Pass
	test_bri_aggregation.py::TestScore	test_score_seismic_hazard_isolated	Pass
	test_bri_aggregation.py::TestScore	test_score_unknown_hazard_returns_zero	Pass
	test_bri_aggregation.py::TestScore	test_score_not_assessed_is_zero	Pass
	test_bri_aggregation.py::TestScore	test_score_verified_is_one	Pass

(continued on next page)

(continued from previous page)

Model	Module	Test Name	Status
	test_consistency_fixes.py::TestPeriodBoundaries	testPeriodBoundaries[1975-1945-1975]	Pass
	test_consistency_fixes.py::TestPeriodBoundaries	testPeriodBoundaries[1976-1976-1999]	Pass
	test_consistency_fixes.py::TestPeriodBoundaries	testPeriodBoundaries[1999-1976-1999]	Pass
	test_consistency_fixes.py::TestPeriodBoundaries	testPeriodBoundaries[2000-2000-2008]	Pass
	test_consistency_fixes.py::TestPeriodBoundaries	testPeriodBoundaries[2008-2000-2008]	Pass
	test_consistency_fixes.py::TestPeriodBoundaries	testPeriodBoundaries[2009-2009-Present]	Pass
	test_consistency_fixes.py::TestPeriodBoundaries	testPeriodBoundaries[2025-2009-Present]	Pass
	test_consistency_fixes.py::TestYearAsString	testYearAsString	Pass
	test_consistency_fixes.py::TestYearInSeconds	testYearInSeconds.with_time_component	Pass
	test_consistency_fixes.py::TestYearInSeconds	testYearInSeconds.input	Pass
	test_consistency_fixes.py::TestYearInSeconds	testYearInSeconds.date_returns_positive_fraction	Pass
	test_flood_resilience_model.py::TestApplyFloodResilienceScore	testApplyFloodResilienceScore.property_id	Pass
	test_flood_resilience_model.py::TestApplyFloodResilienceScore	testApplyFloodResilienceScore.give_different_jitter	Pass
	test_flood_resilience_model.py::TestApplyFloodResilienceScore	testApplyFloodResilienceScore.in_band	Pass
	test_flood_resilience_model.py::TestApplyFloodResilienceScore	testApplyFloodResilienceScore.four	Pass
	test_flood_resilience_model.py::TestApplyFloodResilienceScore	testApplyFloodResilienceScore.damage_modifier	Pass
	test_flood_resilience_model.py::TestApplyFloodResilienceScore	testApplyFloodResilienceScore.to_1_scale	Pass
	test_flood_resilience_model.py::TestApplyFloodResilienceScore	testApplyFloodResilienceScore.three_jittered_scores	Pass
	test_flood_resilience_model.py::TestComputeCampSystemElevationUses51	testComputeCampSystemElevationUses51	Pass
	test_flood_resilience_model.py::TestComputeFloodDamage	testComputeFloodDamage.missing_is_clear	Pass
	test_flood_resilience_model.py::TestComputeFloodDamage	testComputeFloodDamage.uses_flood_damage_severity[Minor damage-0.75]	Pass
	test_flood_resilience_model.py::TestComputeFloodDamage	testComputeFloodDamage.uses_flood_damage_severity[Moderate damage-0.5]	Pass
	test_flood_resilience_model.py::TestComputeFloodDamage	testComputeFloodDamage.uses_flood_damage_severity[No damage-1.0]	Pass
	test_flood_resilience_model.py::TestComputeFloodDamage	testComputeFloodDamage.uses_flood_damage_severity[Severe damage-0.0]	Pass
	test_flood_resilience_model.py::TestComputeFloodDamage	testComputeFloodDamage.uses_flood_damage_severity[Significant damage-0.25]	Pass
	test_flood_resilience_model.py::TestComputeFloodDamage	testComputeFloodDamage.uses_basement_strategy[Deliberately floodable with protection-0.75]	Pass
	test_flood_resilience_model.py::TestComputeFloodDamage	testComputeFloodDamage.uses_basement_strategy[Flood-resistant basement-1.0]	Pass
	test_flood_resilience_model.py::TestComputeFloodDamage	testComputeFloodDamage.uses_basement_strategy[No basement-1.0]	Pass
	test_flood_resilience_model.py::TestComputeFloodDamage	testComputeFloodDamage.uses_basement_strategy[None-0.0]	Pass
	test_flood_resilience_model.py::TestComputeFloodDamage	testComputeFloodDamage.elevation_reads_floor_heights	Pass
	test_flood_resilience_model.py::TestComputeFloodDamage	testComputeFloodDamage.missing_returns_default	Pass
	test_flood_resilience_model.py::TestComputeFloodDamage	testComputeFloodDamage.backup_power_is_min_of_batteries	Pass
	test_flood_resilience_model.py::TestComputeFloodDamage	testComputeFloodDamage.raises	Pass
	test_flood_resilience_model.py::TestComputeFloodDamage	testComputeFloodDamage.roof_floor_hits_flood_capacity	Pass
	test_flood_resilience_model.py::TestComputeFloodDamage	testComputeFloodDamage.high_score_max_scores	Pass
	test_flood_resilience_model.py::TestComputeFloodDamage	testComputeFloodDamage.score	Pass

(continued on next page)

(continued from previous page)

Model	Module	Test Name	Status
test_flood_resilience_model.py::TestDamModel	test_flood_resilience_model.py::TestDamModel	TestDamModelMonotonic	Pass
test_flood_resilience_model.py::TestDamscoreModel	test_flood_resilience_model.py::TestDamscoreModel	TestDamscoreModel gives max mitigation [coastal]	Fail -0.5
test_flood_resilience_model.py::TestDamscoreModel	test_flood_resilience_model.py::TestDamscoreModel	TestDamscoreModel gives max mitigation [fluvial]	Fail -0.6
test_flood_resilience_model.py::TestDamscoreModel	test_flood_resilience_model.py::TestDamscoreModel	TestDamscoreModel gives max mitigation [pluvial]	Fail -0.65
test_flood_resilience_model.py::TestDamscoreModel	test_flood_resilience_model.py::TestDamscoreModel	TestDamscoreModel gives no mitigation [coastal]	Pass 5
test_flood_resilience_model.py::TestDamscoreModel	test_flood_resilience_model.py::TestDamscoreModel	TestDamscoreModel gives no mitigation [fluvial]	Pass 6
test_flood_resilience_model.py::TestDamscoreModel	test_flood_resilience_model.py::TestDamscoreModel	TestDamscoreModel gives no mitigation [pluvial]	Pass 65
test_flood_resilience_model.py::TestDamageModel	test_flood_resilience_model.py::TestDamageModel	TestDamageModel regime defaults to fluvial	Pass
test_flood_resilience_model.py::TestElevationBoundaries	test_flood_resilience_model.py::TestElevationBoundaries	TestElevationBoundaries [-1.0-0.0]	Pass
test_flood_resilience_model.py::TestElevationBoundaries	test_flood_resilience_model.py::TestElevationBoundaries	TestElevationBoundaries [0.0-0.2]	Pass
test_flood_resilience_model.py::TestElevationBoundaries	test_flood_resilience_model.py::TestElevationBoundaries	TestElevationBoundaries [0.5-0.2]	Pass
test_flood_resilience_model.py::TestElevationBoundaries	test_flood_resilience_model.py::TestElevationBoundaries	TestElevationBoundaries [0.999-0.2]	Pass
test_flood_resilience_model.py::TestElevationBoundaries	test_flood_resilience_model.py::TestElevationBoundaries	TestElevationBoundaries [1.0-0.5]	Pass
test_flood_resilience_model.py::TestElevationBoundaries	test_flood_resilience_model.py::TestElevationBoundaries	TestElevationBoundaries [15.0-1.0]	Pass
test_flood_resilience_model.py::TestElevationBoundaries	test_flood_resilience_model.py::TestElevationBoundaries	TestElevationBoundaries [2.5-0.5]	Pass
test_flood_resilience_model.py::TestElevationBoundaries	test_flood_resilience_model.py::TestElevationBoundaries	TestElevationBoundaries [3.0-0.75]	Pass
test_flood_resilience_model.py::TestElevationBoundaries	test_flood_resilience_model.py::TestElevationBoundaries	TestElevationBoundaries [4.99-0.75]	Pass
test_flood_resilience_model.py::TestElevationBoundaries	test_flood_resilience_model.py::TestElevationBoundaries	TestElevationBoundaries [5.0-0.9]	Pass
test_flood_resilience_model.py::TestElevationBoundaries	test_flood_resilience_model.py::TestElevationBoundaries	TestElevationBoundaries [5.99-0.9]	Pass
test_flood_resilience_model.py::TestElevationBoundaries	test_flood_resilience_model.py::TestElevationBoundaries	TestElevationBoundaries [6.0-1.0]	Pass
test_flood_resilience_model.py::TestElevationBoundaries	test_flood_resilience_model.py::TestElevationBoundaries	TestElevationBoundaries returns default	Pass
test_flood_resilience_model.py::TestElevationBoundaries	test_flood_resilience_model.py::TestElevationBoundaries	TestElevationBoundaries is default	Pass
test_flood_resilience_model.py::TestRegimeMap	test_flood_resilience_model.py::TestRegimeMap	TestRegimeMap regime used when field missing	Fail
test_flood_resilience_model.py::TestRegimeMapping	test_flood_resilience_model.py::TestRegimeMapping	TestRegimeMapping risk type maps correctly [Coastal-coastal]	Pass
test_flood_resilience_model.py::TestRegimeMapping	test_flood_resilience_model.py::TestRegimeMapping	TestRegimeMapping risk type maps correctly [Fluvial-fluvial]	Pass
test_flood_resilience_model.py::TestRegimeMapping	test_flood_resilience_model.py::TestRegimeMapping	TestRegimeMapping risk type maps correctly [GroundWater-pluvial]	Pass
test_flood_resilience_model.py::TestRegimeMapping	test_flood_resilience_model.py::TestRegimeMapping	TestRegimeMapping risk type maps correctly [Multiple-pluvial]	Pass
test_flood_resilience_model.py::TestRegimeMapping	test_flood_resilience_model.py::TestRegimeMapping	TestRegimeMapping risk type maps correctly [Pluvial-pluvial]	Pass
test_flood_resilience_model.py::TestRegimeMapOverride	test_flood_resilience_model.py::TestRegimeMapOverride	TestRegimeMapOverride regime override	Pass
test_flood_resilience_model.py::TestRegimeMapFallback	test_flood_resilience_model.py::TestRegimeMapFallback	TestRegimeMapFallback regime falls back to default	Pass
test_flood_resilience_model.py::TestSoftCap	test_flood_resilience_model.py::TestSoftCap	TestSoftCap critical systems zero caps at 60	Pass
test_flood_resilience_model.py::TestSoftCap	test_flood_resilience_model.py::TestSoftCap	TestSoftCap elevation zero caps at 40	Pass
test_flood_resilience_model.py::TestSoftCap	test_flood_resilience_model.py::TestSoftCap	TestSoftCap coastal lower level zero caps at 65 [coastal]	Pass
test_flood_resilience_model.py::TestSoftCap	test_flood_resilience_model.py::TestSoftCap	TestSoftCap coastal lower level zero caps at 65 [fluvial]	Pass
test_flood_resilience_model.py::TestSoftCap	test_flood_resilience_model.py::TestSoftCap	TestSoftCap when all above zero	Pass
test_flood_resilience_model.py::TestSoftCap	test_flood_resilience_model.py::TestSoftCap	TestSoftCap drainage double zero caps at 55	Pass
test_flood_resilience_model.py::TestSoftCap	test_flood_resilience_model.py::TestSoftCap	TestSoftCap clipped to minimum 1	Pass
test_flood_resilience_model.py::TestSpillPhase	test_flood_resilience_model.py::TestSpillPhase	TestSpillPhase per regime	Pass
test_flood_resilience_model.py::TestSpillGenWeights	test_flood_resilience_model.py::TestSpillGenWeights	TestSpillGenWeights sum to 100	Pass
test_flood_resilience_model.py::TestSpillRegions	test_flood_resilience_model.py::TestSpillRegions	TestSpillRegions multipliers cover every measure [coastal]	Pass
test_flood_resilience_model.py::TestSpillRegions	test_flood_resilience_model.py::TestSpillRegions	TestSpillRegions multipliers cover every measure [fluvial]	Pass
test_flood_resilience_model.py::TestSpillRegions	test_flood_resilience_model.py::TestSpillRegions	TestSpillRegions multipliers cover every measure [pluvial]	Pass
test_flood_resilience_model.py::TestSpillGenMeasures	test_flood_resilience_model.py::TestSpillGenMeasures	TestSpillGenMeasures measures	Pass
test_resilience_generator.py::TestResilienceGenerator	test_resilience_generator.py::TestResilienceGenerator	TestResilienceGenerator property can pick other strategies	Pass

(continued on next page)

(continued from previous page)

[illegible]

(continued on next page)

(continued from previous page)

Model	Module	Test Name	Status
	test_schema_invariants.py::TestResiliency	TestResiliencyResiliency_field_uses_resiliency_levels	Pass
	test_schema_invariants.py::TestResiliency	TestResiliencyResiliency_field_uses_resiliency_levels	Pass
	test_schema_invariants.py::TestSchemaSize	TestSchemaSizeSchema_size_at_least_200_leaf_fields	Pass
	test_schema_invariants.py::TestLevelRegion	TestLevelRegionLevel_region_performance_sub_sections	Pass
	test_schema_invariants.py::TestLevelSection	TestLevelSectionLevel_section_level_sections	Pass
	test_schema_invariants.py::TestLevelSection	TestLevelSectionLevel_section_and_incidents_sub_sections	Pass
	test_schema_invariants.py::TestLevelSection	TestLevelSectionLevel_section_header_sub_sections	Pass
	test_schema_invariants.py::TestLevelSection	TestLevelSectionLevel_section_measures_risk_assessment	Pass
	test_schema_invariants.py::TestLevelSection	TestLevelSectionLevel_section_measures_sub_sections	Pass
	test_schema_invariants.py::TestLevelSection	TestLevelSectionLevel_section_measures_five_buckets	Pass
	test_schema_invariants.py::TestLevelSection	TestLevelSectionLevel_section_history_sub_sections	Pass

3 Building Resilience Index Model (MKM-BRI-001)

Table 4: Test Results Summary — Building Resilience Index Model (MKM-BRI-001)

Total Tests	202
Passed	202
Failed	0
Skipped	0
Duration	0.21s

Table 5: Detailed Test Results — Building Resilience Index Model

Test Description	Acceptance Criteria	Result	Time
Ratings and metadata are written	<code>gbr["BRIRating"] in ("AA", "AA+", "A", "A+", "B", "B+", "...;</code> <code>gbr["BRIRatingVersion"] == "BRI v1.6" (+3 more)</code>	Pass	0.000s
A downstream methodology owns the score fields; this helper must not Agent optional	<code>k not in gbr</code> <code>"BRIRatingAgent" not in gbr or gbr["BRIRatingAgent"] is None</code>	Pass	0.000s
Rating date override	<code>gbr["BRIDate"] == "2024-01-15"</code>	Pass	0.000s
Version override	<code>gbr["BRIRatingVersion"] == "BRI v2.0-alpha"</code>	Pass	0.000s
Deterministic for same property id	<code>gbr1["BRIFloodScore"] == gbr2["BRIFloodScore"];</code> <code>gbr1["BRIWindScore"] == gbr2["BRIWindScore"]</code>	Pass	0.000s
Different property ids give different jitter	<code>gbr1["BRIFloodScore"] == gbr2["BRIFloodScore"];</code> <code>gbr1["BRIWindScore"] != gbr2["BRIWindScore"] ...</code>	Pass	0.000s
Jittered scores within band	<code>abs(gbr[f"BRI{h}Score"] - flood) <= 0.051</code>	Pass	0.000s
Overall is mean of four	<code>gbr["BRIScore"] == expected</code>	Pass	0.000s
Return value includes damage modifier	<code>"damage_modifier" in result;</code> <code>0.0 < result["damage_modifier"] <= 1.0 (+3 more)</code>	Pass	0.000s
Score persisted on 0 to 1 scale	<code>0.0 <= gbr["BRIFloodScore"] <= 1.0;</code> <code>gbr["BRIFloodScore"] > 0.9</code>	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Writes flood and three jittered scores	"BRIFloodScore" in gbr; f"BRI{hazard}Score" in gbr	Pass	0.000s
Metadata fields present	k in gbr	Pass	0.000s
Overall bri rating menu	gbr["BRIRating"]["type"] == "menu"; gbr["BRIRating"]["options"] == ["AA", "AA+", "A", "A+", "...	Pass	0.000s
Overall bri score decimal	gbr["BRIScore"]["type"] == "decimal"	Pass	0.000s
Per hazard rating[fire] [hazard='Fire']	f["type"] == "menu"; f["options"] == ["AA", "A", "B", "NR", "N/A"]	Pass	0.000s
Per hazard rating[flood] [hazard='Flood']	f["type"] == "menu"; f["options"] == ["AA", "A", "B", "NR", "N/A"]	Pass	0.000s
Per hazard rating[seismic] [hazard='Seismic']	f["type"] == "menu"; f["options"] == ["AA", "A", "B", "NR", "N/A"]	Pass	0.000s
Per hazard rating[wind] [hazard='Wind']	f["type"] == "menu"; f["options"] == ["AA", "A", "B", "NR", "N/A"]	Pass	0.000s
Per hazard score[fire] [hazard='Fire']	f["type"] == "decimal"	Pass	0.000s
Per hazard score[flood] [hazard='Flood']	f["type"] == "decimal"	Pass	0.000s
Per hazard score[seismic] [hazard='Seismic']	f["type"] == "decimal"	Pass	0.000s
Per hazard score[wind] [hazard='Wind']	f["type"] == "decimal"	Pass	0.000s
Basement property can pick other strategies	observed != {"No basement"}	Pass	0.004s
No basement property always no basement	r["SiteAndDrainage"]["BasementFloodStrategy"] == "No base..."	Pass	0.001s
Distribution summary counts	summary["AA"]["actual"] == pytest.approx(0.1); summary["A"]["actual"] == pytest.approx(0.2) (+3 more)	Pass	0.000s
Distribution summary empty	summary[k]["actual"] == 0.0	Pass	0.000s
Recalibrate empty returns current	thresholds == RATING.THRESHOLDS	Pass	0.000s
Recalibrate thresholds hits target	thresholds["AA"] > thresholds["A"] > thresholds["B"]	Pass	0.000s
All not assessed is nr	r["rating"] == "NR"	Pass	0.000s
All verified no hazard profile is top rating	r["rating"] in ("AA", "AA+"); r["score"] == pytest.approx(1.0)	Pass	0.000s
Empty input	r["rating"] == "NR"; r["score"] == 0.0	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
None hazards excluded from weakest link	<code>result["hazard_ratings"]["Wind"] == "N/A"; result["rating"] in ("AA", "AA+")</code>	Pass	0.000s
Plus modifier not applied to nr	<code>"+" not in result["rating"]</code>	Pass	0.000s
Plus modifier when continuity strong	<code>result["rating"] == "AA+"</code>	Pass	0.000s
Returns full breakdown	<code>set(r) == {"rating", "score", "section_scores", "hazard_r..."; set(r["section_scores"]) == set(SECTION_WEIGHTS) (+2 more)</code>	Pass	0.000s
Weakest link picks worst hazard	<code>result["hazard_ratings"]["Flood"] == "NR"; result["rating"] == "NR"</code>	Pass	0.000s
Critical systems elevation uses 5level	<code>compute_compliance("critical_systems_elevation", rec) == 1.0; compute_compliance("critical_systems_elevation", rec) == 0.0 (+1 more)</code>	Pass	0.000s
No flood event recorded → property is in the 'clear' state.	<code>compute_compliance("historic_water_area_flag", rec) == 1.0</code>	Pass	0.000s
Historic water uses flood damage severity[minor damage-0.75] [severity='Minor damage', expected=0.75]	<code>compute_compliance("historic_water_area_flag", rec) == ex...</code>	Pass	0.000s
Historic water uses flood damage severity[moderate damage-0.5] [severity='Moderate damage', expected=0.5]	<code>compute_compliance("historic_water_area_flag", rec) == ex...</code>	Pass	0.000s
Historic water uses flood damage severity[no damage-1.0] [severity='No damage', expected=1.0]	<code>compute_compliance("historic_water_area_flag", rec) == ex...</code>	Pass	0.000s
Historic water uses flood damage severity[severe damage-0.0] [severity='Severe damage', expected=0.0]	<code>compute_compliance("historic_water_area_flag", rec) == ex...</code>	Pass	0.000s
Historic water uses flood damage severity[significant damage-0.25] [severity='Significant damage', expected=0.25]	<code>compute_compliance("historic_water_area_flag", rec) == ex...</code>	Pass	0.000s
Lower level design uses basement strategy[deliberately floodable with protection-0.75] [strategy='Deliberately floodable with protection', expected=0.75]	<code>compute_compliance("lower_level_flood_compatibility_design", rec) == ex...</code>	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Lower level design uses basement strategy[flood-resistant basement-1.0] [strategy='Flood-resistant basement', expected=1.0]	compute_compliance("lower_level_flood_compatibility_design")	Pass	0.000s.
Lower level design uses basement strategy[no basement-1.0] [strategy='No basement', expected=1.0]	compute_compliance("lower_level_flood_compatibility_design")	Pass	0.000s.
Lower level design uses basement strategy[none-0.0] [strategy='None', expected=0.0]	compute_compliance("lower_level_flood_compatibility_design")	Pass	0.000s.
Lowest floor elevation reads floor meters	compute_compliance("lowest_occupied_floor_elevation")	Pass	0.000s
Lowest floor missing returns default	compute_compliance("lowest_occupied_floor_elevation")	Pass	0.000s
Spec measure is min(SumpPump credit, BackupPower credit).	compute_compliance("sump_pumps_backup_power_credit")	Pass	0.000s
Unknown measure raises	—	Pass	0.000s
All not assessed zero floor hits floor cap	result["score_final"] <= 40.0; result["score_final"] >= 1.0	Pass	0.000s
All verified high floor max scores	result["score_raw"] == pytest.approx(100.0, abs=0.5); result["score_final"] == pytest.approx(100.0, abs=0.5) (+2 more)	Pass	0.000s
Compliance dict keys	set(result["compliance"]) == set(FLOOD_SPEC_BASE_WEIGHTS)	Pass	0.000s
Bungalow forced to single storey County lookup[henley-on-thames-oxfordshire] [town='Henley-on-Thames', expected_county='Oxfordshire']	rec["PropertyHeader"]["PropertyAttributes"] ["County"] == expected.c...	Pass	0.000s
County lookup[london-greater london] [town='London', expected_county='Greater London']	rec["PropertyHeader"]["Location"] ["County"] == expected.c...	Pass	0.000s
County lookup[oxford-oxfordshire] [town='Oxford', expected_county='Oxfordshire']	rec["PropertyHeader"]["Location"] ["County"] == expected.c...	Pass	0.000s
County lookup[reading-berkshire] [town='Reading', expected_county='Berkshire']	rec["PropertyHeader"]["Location"] ["County"] == expected.c...	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
County lookup[slough-berkshire] [town='Slough', expected_county='Berkshire']	rec["PropertyHeader"]["Location"]["County"] == expected.c...	Pass	0.000s
County lookup[westminster-greater london] [town='Westminster', expected_county='Greater London']	rec["PropertyHeader"]["Location"]["County"] == expected.c...	Pass	0.000s
Never rented clears rental fields	rec["TransactionHistory"]["Rental"]["MonthRentedGbp"] is ...; rec["TransactionHistory"]["Rental"]["RentalYield"] is None	Pass	0.000s
No damage clears last flood date	rec["HistoryAndIncidents"]["FloodEvents"][-1]["LastFloodDate"]	Pass	0.000s
None fire severity leaves date alone	rec["HistoryAndIncidents"]["FireIncidents"][-1]["LastFireDate"]	Pass	0.000s
Overall flood risk from zone 1-low [zone='Zone 1', expected='Low']	rec["PropertyHeader"]["RiskAssessment"][-1]["OverallFloodRisk"]	Pass	0.000s
Overall flood risk from zone 2-medium [zone='Zone 2', expected='Medium']	rec["PropertyHeader"]["RiskAssessment"][-1]["OverallFloodRisk"]	Pass	0.000s
Overall flood risk from zone 3a-high [zone='Zone 3a', expected='High']	rec["PropertyHeader"]["RiskAssessment"][-1]["OverallFloodRisk"]	Pass	0.000s
Overall flood risk from zone 3b-very high [zone='Zone 3b', expected='Very high']	rec["PropertyHeader"]["RiskAssessment"][-1]["OverallFloodRisk"]	Pass	0.000s
Property height synced to height meters	rec["PropertyHeader"]["Construction"]["PropertyHeight"] == ...	Pass	0.000s
Property period derived from construction year	rec["PropertyHeader"]["PropertyAttributes"][-1]["PropertyPeriod"]	Pass	0.000s
Purchase price discounted by years since	800.000 < pp < 850.000	Pass	0.000s
Rental yield derived	rec["TransactionHistory"]["Rental"]["RentalYield"] == pyt...	Pass	0.000s
Sale price discounted by years since	450.000 < sp < 475.000	Pass	0.000s
Severe fire gets a date	d is not None and len(d) == 10	Pass	0.000s
Unknown town keeps county	rec["PropertyHeader"]["Location"]["County"] == "Original ...	Pass	0.000s
Modifier monotonic	damage_modifier(75.0, "fluvial") < damage_modifier(50.0, ...; damage_modifier(50.0, "fluvial") < damage_modifier(25.0, ...	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Score 100 gives max mitigation[coastal-0.5] [regime='coastal', alpha=0.5]	damage_modifier(100.0, regime) == round(1.0 - alpha, 4)	Pass	0.000s
Score 100 gives max mitigation[fluvial-0.6] [regime='fluvial', alpha=0.6]	damage_modifier(100.0, regime) == round(1.0 - alpha, 4)	Pass	0.000s
Score 100 gives max mitigation[pluvial-0.65] [regime='pluvial', alpha=0.65]	damage_modifier(100.0, regime) == round(1.0 - alpha, 4)	Pass	0.000s
Score 1 gives no mitigation[coastal-0.5] [regime='coastal', alpha=0.5]	damage_modifier(1.0, regime) == 1.0	Pass	0.000s
Score 1 gives no mitigation[fluvial-0.6] [regime='fluvial', alpha=0.6]	damage_modifier(1.0, regime) == 1.0	Pass	0.000s
Score 1 gives no mitigation[pluvial-0.65] [regime='pluvial', alpha=0.65]	damage_modifier(1.0, regime) == 1.0	Pass	0.000s
Unknown regime defaults to fluvial	damage_modifier(50.0, "bogus") == damage_modifier(50.0, "...")	Pass	0.000s
Band boundaries[-1.0-0.0] [meters=-1.0, expected=0.0]	_elevation_to_compliance(meters) == expected	Pass	0.000s
Band boundaries[0.0-0.2] [meters=0.0, expected=0.2]	_elevation_to_compliance(meters) == expected	Pass	0.000s
Band boundaries[0.5-0.2] [meters=0.5, expected=0.2]	_elevation_to_compliance(meters) == expected	Pass	0.000s
Band boundaries[0.999-0.2] [meters=0.999, expected=0.2]	_elevation_to_compliance(meters) == expected	Pass	0.000s
Band boundaries[1.0-0.5] [meters=1.0, expected=0.5]	_elevation_to_compliance(meters) == expected	Pass	0.000s
Band boundaries[15.0-1.0] [meters=15.0, expected=1.0]	_elevation_to_compliance(meters) == expected	Pass	0.000s
Band boundaries[2.5-0.5] [meters=2.5, expected=0.5]	_elevation_to_compliance(meters) == expected	Pass	0.000s
Band boundaries[3.0-0.75] [meters=3.0, expected=0.75]	_elevation_to_compliance(meters) == expected	Pass	0.000s
Band boundaries[4.99-0.75] [meters=4.99, expected=0.75]	_elevation_to_compliance(meters) == expected	Pass	0.000s
Band boundaries[5.0-0.9] [meters=5.0, expected=0.9]	_elevation_to_compliance(meters) == expected	Pass	0.000s
Band boundaries[5.99-0.9] [meters=5.99, expected=0.9]	_elevation_to_compliance(meters) == expected	Pass	0.000s
Band boundaries[6.0-1.0] [meters=6.0, expected=1.0]	_elevation_to_compliance(meters) == expected	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Non numeric returns default	<code>_elevation_to_compliance("not a number") == DEFAULT_MISSI...</code>	Pass	0.000s
None returns default	<code>_elevation_to_compliance(None) == DEFAULT_MISSING_COMPLIANCE</code>	Pass	0.000s
Basement flood strategy[deliberately floodable with protection-0.75] [value='Deliberately floodable with protection', expected=0.75]	<code>_field_credit("BasementFloodStrategy", value) == expected</code>	Pass	0.000s
Basement flood strategy[flood-resistant basement-1.0] [value='Flood-resistant basement', expected=1.0]	<code>_field_credit("BasementFloodStrategy", value) == expected</code>	Pass	0.000s
Basement flood strategy[no basement-1.0] [value='No basement', expected=1.0]	<code>_field_credit("BasementFloodStrategy", value) == expected</code>	Pass	0.000s
Basement flood strategy[none-0.0] [value='None', expected=0.0]	<code>_field_credit("BasementFloodStrategy", value) == expected</code>	Pass	0.000s
Boolean false zero credit	<code>_field_credit("AnyField", False) == 0.0</code>	Pass	0.000s
Boolean true full credit	<code>_field_credit("AnyField", True) == 1.0</code>	Pass	0.000s
None value is zero	<code>_field_credit("anything", None) == 0.0</code>	Pass	0.000s
Resilience levels[enhanced-0.9] [level='Enhanced', expected=0.9]	<code>_field_credit("AnyField", level) == expected</code>	Pass	0.000s
Resilience levels[meets minimum-0.7] [level='Meets minimum', expected=0.7]	<code>_field_credit("AnyField", level) == expected</code>	Pass	0.000s
Resilience levels[not assessed-0.0] [level='Not assessed', expected=0.0]	<code>_field_credit("AnyField", level) == expected</code>	Pass	0.000s
Resilience levels[partial-0.4] [level='Partial', expected=0.4]	<code>_field_credit("AnyField", level) == expected</code>	Pass	0.000s
Resilience levels[verified-1.0] [level='Verified', expected=1.0]	<code>_field_credit("AnyField", level) == expected</code>	Pass	0.000s
Unknown string zero	<code>_field_credit("AnyField", "Made up value") == 0.0</code>	Pass	0.000s
Excellent condition beats very poor at fixed period	<code>self._scored_score(excellent) > self._scored_score(very.p...</code>	Pass	0.032s

Test Description	Acceptance Criteria	Result	Time
Flood zone uplift helps flood protection	<code>avg_flood(zone3a) > avg_flood(zone1)</code>	Pass	0.032s
2009-Present should score higher than 1945-1975 at the same condition.	<code>modern > postwar</code>	Pass	0.033s
All values in valid menu	<code>value in { "None", "No basement",...;</code> <code>value in RESILIENCE_LEVEL_CREDIT</code>	Pass	0.000s
Deterministic with seeded rng	<code>generate_resilience(prop, rng1) ==</code> <code>generate_resilience(pr...</code>	Pass	0.000s
Emits all five sub sections	<code>set(r) == { "BuildingAssessment",</code> <code>"SiteAndDra...</code>	Pass	0.000s
Missing hazard profile defaults all applicable	<code>gbr[f"BRI{h}Rating"] != "N/A"</code>	Pass	0.000s
None hazard class produces na	<code>gbr["BRIFloodRating"] != "N/A";</code> <code>gbr["BRIWindRating"] == "N/A" (+2</code> <code>more)</code>	Pass	0.000s
Hazard class options[fire] [hazard='Fire']	<code>f["type"] == "menu"; f["options"] ==</code> <code>["None", "Low", "Medium", "High",</code> <code>"Extreme"]</code>	Pass	0.000s
Hazard class options[flood] [hazard='Flood']	<code>f["type"] == "menu"; f["options"] ==</code> <code>["None", "Low", "Medium", "High",</code> <code>"Extreme"]</code>	Pass	0.000s
Hazard class options[seismic] [hazard='Seismic']	<code>f["type"] == "menu"; f["options"] ==</code> <code>["None", "Low", "Medium", "High",</code> <code>"Extreme"]</code>	Pass	0.000s
Hazard class options[wind] [hazard='Wind']	<code>f["type"] == "menu"; f["options"] ==</code> <code>["None", "Low", "Medium", "High",</code> <code>"Extreme"]</code>	Pass	0.000s
All types are valid	<code>fdef["type"] in _VALID_TYPES</code>	Pass	0.000s
Every leaf has description	<code>"description" in fdef;</code> <code>isinstance(fdef["description"],</code> <code>str) (+1 more)</code>	Pass	0.000s
Menu fields have options	<code>"options" in fdef;</code> <code>isinstance(fdef["options"], list)</code> <code>(+1 more)</code>	Pass	0.000s
Bucket boundaries[-0.1-not assessed] [score=-0.1, expected='Not assessed']	<code>_level_from_score(score) == expected</code>	Pass	0.000s
Bucket boundaries[0.0-not assessed] [score=0.0, expected='Not assessed']	<code>_level_from_score(score) == expected</code>	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Bucket boundaries[0.19-not assessed] [score=0.19, expected='Not assessed']	<code>_level_from_score(score) == expected</code>	Pass	0.000s
Bucket boundaries[0.2-partial] [score=0.2, expected='Partial']	<code>_level_from_score(score) == expected</code>	Pass	0.000s
Bucket boundaries[0.39-partial] [score=0.39, expected='Partial']	<code>_level_from_score(score) == expected</code>	Pass	0.000s
Bucket boundaries[0.4-meets minimum] [score=0.4, expected='Meets minimum']	<code>_level_from_score(score) == expected</code>	Pass	0.000s
Bucket boundaries[0.59-meets minimum] [score=0.59, expected='Meets minimum']	<code>_level_from_score(score) == expected</code>	Pass	0.000s
Bucket boundaries[0.6-enhanced] [score=0.6, expected='Enhanced']	<code>_level_from_score(score) == expected</code>	Pass	0.000s
Bucket boundaries[0.79-enhanced] [score=0.79, expected='Enhanced']	<code>_level_from_score(score) == expected</code>	Pass	0.000s
Bucket boundaries[0.8-verified] [score=0.8, expected='Verified']	<code>_level_from_score(score) == expected</code>	Pass	0.000s
Bucket boundaries[1.0-verified] [score=1.0, expected='Verified']	<code>_level_from_score(score) == expected</code>	Pass	0.000s
Bucket boundaries[1.5-verified] [score=1.5, expected='Verified']	<code>_level_from_score(score) == expected</code>	Pass	0.000s
A threshold	<code>_map_score_to_rating(RATING_THRESHOLDS["A"]) == "A";</code> <code>_map_score_to_rating(RATING_THRESHOLDS["AA"] - 0.001) == "A"</code>	Pass	0.000s
Aa threshold	<code>_map_score_to_rating(RATING_THRESHOLDS["AA"]) == "AA";</code> <code>_map_score_to_rating(1.0) == "AA"</code>	Pass	0.000s
B threshold	<code>_map_score_to_rating(RATING_THRESHOLDS["B"]) == "B"</code>	Pass	0.000s
Below b is nr	<code>_map_score_to_rating(0.0) == "NR";</code> <code>_map_score_to_rating(RATING_THRESHOLDS["B"] - 0.001) == "NR"</code>	Pass	0.000s
Boundaries[1850-pre-1919] [year=1850, expected='Pre-1919']	<code>_period_from_year(year) == expected</code>	Pass	0.000s
Boundaries[1918-pre-1919] [year=1918, expected='Pre-1919']	<code>_period_from_year(year) == expected</code>	Pass	0.000s
Boundaries[1919-1919-1944] [year=1919, expected='1919-1944']	<code>_period_from_year(year) == expected</code>	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Boundaries[1944-1919-1944] [year=1944, expected='1919-1944']	<code>.period_from_year(year) == expected</code>	Pass	0.000s
Boundaries[1945-1945-1975] [year=1945, expected='1945-1975']	<code>.period_from_year(year) == expected</code>	Pass	0.000s
Boundaries[1975-1945-1975] [year=1975, expected='1945-1975']	<code>.period_from_year(year) == expected</code>	Pass	0.000s
Boundaries[1976-1976-1999] [year=1976, expected='1976-1999']	<code>.period_from_year(year) == expected</code>	Pass	0.000s
Boundaries[1999-1976-1999] [year=1999, expected='1976-1999']	<code>.period_from_year(year) == expected</code>	Pass	0.000s
Boundaries[2000-2000-2008] [year=2000, expected='2000-2008']	<code>.period_from_year(year) == expected</code>	Pass	0.000s
Boundaries[2008-2000-2008] [year=2008, expected='2000-2008']	<code>.period_from_year(year) == expected</code>	Pass	0.000s
Boundaries[2009-2009-present] [year=2009, expected='2009-Present']	<code>.period_from_year(year) == expected</code>	Pass	0.000s
Boundaries[2025-2009-present] [year=2025, expected='2009-Present']	<code>.period_from_year(year) == expected</code>	Pass	0.000s
Default regime used when field missing	<code>result["regime"] == DEFAULT_REGIME</code>	Pass	0.000s
Each flood risk type maps correctly[coastal-coastal] [flood_type='Coastal', regime='coastal']	<code>REGIME_FROM_FLOOD_RISK_TYPE[flood_type] == regime</code>	Pass	0.000s
Each flood risk type maps correctly[fluvial-fluvial] [flood_type='Fluvial', regime='fluvial']	<code>REGIME_FROM_FLOOD_RISK_TYPE[flood_type] == regime</code>	Pass	0.000s
Each flood risk type maps correctly[groundwater-pluvial] [flood_type='GroundWater', regime='pluvial']	<code>REGIME_FROM_FLOOD_RISK_TYPE[flood_type] == regime</code>	Pass	0.000s
Each flood risk type maps correctly[multiple-pluvial] [flood_type='Multiple', regime='pluvial']	<code>REGIME_FROM_FLOOD_RISK_TYPE[flood_type] == regime</code>	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Each flood risk type maps correctly[pluvial-pluvial] [flood_type='Pluvial', regime='pluvial']	REGIME_FROM_FLOOD_RISK_TYPE[flood_type] == regime	Pass	0.000s
Explicit regime override	result["regime"] == "coastal"	Pass	0.000s
Invalid regime falls back to default	result["regime"] == DEFAULT_REGIME	Pass	0.000s
Basement flood strategy keeps own vocab	f["options"] == ["None", "No basement", ...	Pass	0.000s
Every resilience field uses resilience levels	offenders == []	Pass	0.000s
Levels are the canonical five	RESILIENCE_LEVELS == ["Not assessed", "Parti...	Pass	0.000s
Even though apply_bri-rating doesn't WRITE scores into the record,	"rating" in result; "score" in result and isinstance(result["score"], float) (+4 more)	Pass	0.000s
The Thames target is 10/20/40/30.	res["sample_size"] == 1000; 0.0 <=	Pass	0.084s
The current weakest-link logic	res["mean_score"] <= 1.0 (+1 more)		
At least 200 leaf fields	len(leaves) >= 200	Pass	0.000s
Hazard with all verified is one	score_hazard("Flood", resilience) == pytest.approx(1.0)	Pass	0.000s
Seismic hazard isolated	score == pytest.approx(1.0)	Pass	0.000s
Unknown hazard returns zero	score_hazard("NotARealHazard", {}) == 0.0	Pass	0.000s
All not assessed is zero	score_section("BuildingAssessment", data) == 0.0	Pass	0.000s
All verified is one	score_section("BuildingAssessment", data) == 1.0	Pass	0.000s
Empty section returns zero	score_section("BuildingAssessment", {}) == 0.0	Pass	0.000s
Mixed levels average correctly	result == pytest.approx(0.5)	Pass	0.000s
Critical systems zero caps at 60	_apply_soft_caps(80.0, comp, "fluvial") == 60.0	Pass	0.000s
Floor elevation zero caps at 40	_apply_soft_caps(80.0, comp, "fluvial") == 40.0	Pass	0.000s
Fluvial coastal lower level zero caps at 65[coastal] [regime='coastal']	_apply_soft_caps(80.0, comp, regime) == 65.0; _apply_soft_caps(80.0, comp, "pluvial") == 80.0	Pass	0.000s
Fluvial coastal lower level zero caps at 65[fluvial] [regime='fluvial']	_apply_soft_caps(80.0, comp, regime) == 65.0; _apply_soft_caps(80.0, comp, "pluvial") == 80.0	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
No cap when all above zero	<code>_apply_soft_caps(85.0, comp, "fluvial") == 85.0</code>	Pass	0.000s
Pluvial drainage double zero caps at 55	<code>_apply_soft_caps(80.0, comp, "pluvial") == 55.0;</code> <code>_apply_soft_caps(80.0, comp, "fluvial") == 80.0</code>	Pass	0.000s
Score clipped to minimum 1	<code>_apply_soft_caps(0.0, comp, "fluvial") == 1.0</code>	Pass	0.000s
Alpha per regime	<code>FLOOD_SPEC.ALPHA_BY_REGIME["pluvial"] == 0.65;</code> <code>FLOOD_SPEC.ALPHA_BY_REGIME["fluvial"] == 0.60 (+1 more)</code>	Pass	0.000s
Base weights sum to 100	<code>sum(FLOOD_SPEC.BASE_WEIGHTS.values()) == 100</code>	Pass	0.000s
Regime multipliers cover every measure[coastal] [regime='coastal']	<code>set(FLOOD_SPEC.REGIME_MULTIPLIERS[regime]) == set(FLOOD_S...</code>	Pass	0.000s
Regime multipliers cover every measure[fluvial] [regime='fluvial']	<code>set(FLOOD_SPEC.REGIME_MULTIPLIERS[regime]) == set(FLOOD_S...</code>	Pass	0.000s
Regime multipliers cover every measure[pluvial] [regime='pluvial']	<code>set(FLOOD_SPEC.REGIME_MULTIPLIERS[regime]) == set(FLOOD_S...</code>	Pass	0.000s
Thirteen measures	<code>len(FLOOD_SPEC.BASE_WEIGHTS) == 13</code>	Pass	0.000s
Energy performance sub sections	<code>set(PROPERTY_SCHEMA["EnergyPerformance"]) == { ...</code>	Pass	0.000s
Five top level sections	<code>set(PROPERTY_SCHEMA) == { "PropertyHeader", "...</code>	Pass	0.000s
History and incidents sub sections	<code>set(PROPERTY_SCHEMA["HistoryAndIncidents"]) == { ...</code>	Pass	0.000s
Property header sub sections	<code>set(PROPERTY_SCHEMA["PropertyHeader"]) == { "...</code>	Pass	0.000s
Protection measures risk assessment split	<code>set(ra) == {"InsuranceBodyRatings", "GoverningBodyRatings"}</code>	Pass	0.000s
Protection measures sub sections	<code>set(PROPERTY_SCHEMA["ProtectionMeasures"]) == { ...</code>	Pass	0.000s
Resilience measures five buckets	<code>set(rm) == { "BuildingAssessment", "SiteAndDr...</code>	Pass	0.000s
Transaction history sub sections	<code>set(PROPERTY_SCHEMA["TransactionHistory"]) == { ...</code>	Pass	0.000s
Current weights sum to one	—	Pass	0.000s
Raises on drift	—	Pass	0.000s
Bad string	<code>_years_since("not a date") is None</code>	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Iso with time component	<code>_years_since(d) == pytest.approx(2.0, abs=0.01)</code>	Pass	0.000s
None input	<code>_years_since(None) is None</code>	Pass	0.000s
Recent date returns positive fraction	<code>_years_since(d) == pytest.approx(3.0, abs=0.01)</code>	Pass	0.000s